

SYSTEM ARCHITECTURE DOCUMENT

Employee Management System

Architecture overview of the application as built, the target Next.js 16 + Supabase stack, plus the planned Cash Flow Management module.

Generated 2026-07-08 · Next.js 16 / React 19 / Supabase / TypeScript

1. Overview

Employee Management System is a role-aware enterprise management application covering departments (SBUs), projects, tasks, attendance, daily work logs, a personnel directory, file storage, and reporting. The frontend is built with Next.js; the backend of record is Supabase (Postgres + Auth + Storage), replacing the browser-only localStorage prototype persistence described further below.

Tech Stack

Layer	Choice	Notes
Frontend framework	Next.js 16 (App Router)	Single route <code>/</code> ; view components render client-side (<code>'use client'</code>)
UI library	React 19	Function components + hooks only, no class components
Language	TypeScript 5.9	Strict domain types in <code>lib/types.ts</code>
Styling	Tailwind CSS v4	CSS-first config in <code>app/globals.css</code> ; class-based dark mode via <code>@custom-variant dark</code>
Icons	lucide-react	Consistent icon set across all views
Animation	motion (Framer Motion)	Declared dependency for future transitions
Utilities	clsx, tailwind-merge, class-variance-authority	Conditional / merged class name composition
Backend	Supabase (Postgres, Auth, Storage)	Server-side database, authentication, and file storage for the application
Persistence (current codebase)	Browser <code>localStorage</code>	The code in this repo today still uses the localStorage prototype pattern described in §3; migrating each data model to Supabase tables is the next implementation step
Tooling	ESLint 9 (flat config), tsc	Type checking + linting, no test runner configured

2. Project Structure

```
enterprise-project-&-employee-management/
├─ app/
│   ├── layout.tsx      Root HTML shell, font, metadata
│   ├── page.tsx        Single entry point: state, routing, data CRUD, layout shell
│   └─ globals.css      Tailwind import + dark-mode variant
├─ components/
│   ├── Sidebar.tsx     Collapsible left nav (sections, department filter)
│   ├── Header.tsx      Top bar: search, role switch, theme toggle, notifications
│   ├── CommandPalette.tsx  Ctrl+K global search/jump modal
│   ├── AdminDashboard.tsx   Admin home: KPIs, charts, SBU grid, milestones
│   ├── TeamLeaderDashboard.tsx Team Leader home: team stats, approvals
│   ├── TeamMemberDashboard.tsx Team Member home: my tasks, daily log submission
│   ├── DepartmentsView.tsx  SBU table → project table → task table drill-down
│   ├── ProjectsView.tsx    Project portfolio grid + project detail tabs
│   ├── TasksView.tsx      Global task table (status dropdown, editable %)
│   └─ CalendarView.tsx    Month/Week calendar + daily-works modal
```

		ReportsView.tsx	Cross-department analytics
		UsersView.tsx	Personnel directory table
		FilesView.tsx	Media/document library
		lib/	
		types.ts	All domain TypeScript interfaces
		initialData.ts	Seed data + <code>getStoredData()/saveStoredData()</code> (<code>localStorage</code>)
		utils.ts	Class-name merge helper (<code>cn</code>)
		hooks/	
		use-mobile.ts	Viewport breakpoint hook

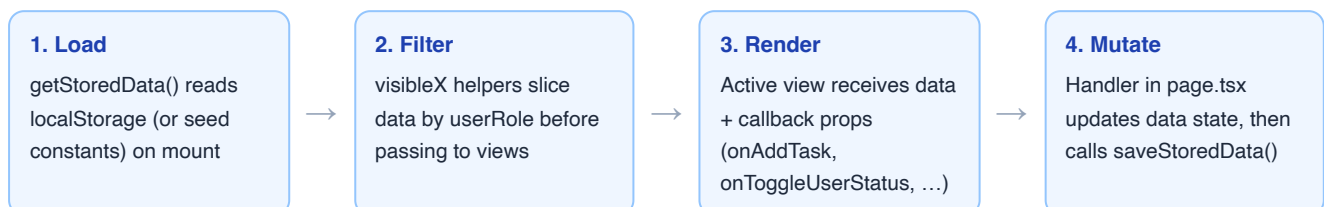
3. Application Architecture

`app/page.tsx` is the sole stateful controller. It owns the full in-memory dataset and all layout state, and conditionally renders one view component at a time based on `currentView`. There is no client-side router, context provider, or external state library — everything is prop-drilled one level from `page.tsx` down to the active view.

Top-level state in page.tsx

- `data` departments, users, projects, tasks, attendance, media, notifications, worklogs
- `currentView` which view component is active (Dashboard, Departments, Tasks, Calendar, Users, Reports, Settings...)
- `userRole` Admin / Team Leader / Team Member — swappable from the Header for demo purposes
- `selectedDeptId / selectedProjectId` drill-down selection shared between Sidebar filter and Departments/Projects views
- `darkMode, sidebarCollapsed, isCommandPaletteOpen` chrome/layout UI state

Data flow (unidirectional)



Every create/update action (add task, change status, edit progress, toggle account status, submit daily log, etc.) is a handler defined in `page.tsx` and passed down as a prop — child views never touch `localStorage` directly. Some views (e.g. Departments) also keep short-lived local UI state for instant optimistic updates before the parent's canonical state re-renders them.

Role-based rendering

The same routes render different dashboards and, in some views, different permissions depending on `userRole`:

- **Admin** — full visibility across all departments; sees AdminDashboard, Users (Personnel Directory) nav item, can edit task progress
- **Team Leader** — sees TeamLeaderDashboard, department/team scoped data, can edit task progress
- **Team Member** — sees TeamMemberDashboard, own-task scoped views, progress is read-only

Backend Migration Plan (Supabase) Target — not yet built

Supabase is the designated backend going forward. The intended shape of the migration:

- A Supabase Postgres table per existing data model (departments, users, projects, tasks, attendance, media, notifications, worklogs), mirroring the interfaces in §4.

- Supabase Auth replaces the current demo-only role switcher in the Header, with `userRole` sourced from an authenticated session/profile row instead of local UI state.
- `getStoredData()` / `saveStoredData()` in `lib/initialData.ts` are replaced by Supabase client queries (`supabase.from(table).select()/insert()/update()`), keeping the same shape of data flowing into `page.tsx` so the view components and their props are largely unaffected.
- Supabase Storage replaces the mocked `MediaFile.url` placeholders for real file uploads (project documents, task attachments, daily-work photos).
- New tables (like Cash Flow, §6) are built Supabase-first rather than being added to the localStorage pattern.

4. Data Model

All interfaces live in `lib/types.ts`. Every record type below is an array on the root `data` object.

User

Field	Type
id, name, email, title	string
role	'Admin' 'Team Leader' 'Team Member'
departmentId	string (FK → Department)
status	'Active' 'Inactive'
avatar	string (image URL)
performanceScore	number (0–100)
phone	string?

Department

Field	Type
id, name, icon, code, description	string

Project

Field	Type
id, name, description, deadline	string
departmentId	string (FK → Department)
status	'Planning' 'In Progress' 'In Review' 'Completed'
progress	number (%)
leaderId	string (FK → User)
members	string[] (FK[] → User)
documents / images / videos	MediaFile[]
notes	string[]

Task

Field	Type
id, name, description	string
projectId, departmentId, assignedTo	string (FKs)
category	'daily' 'continuous'
priority	'High' 'Medium' 'Low'
status	'Todo' 'In Progress' 'Review' 'Completed' 'Cancelled'
progress	number (0–100), directly editable by Admin/Team Leader
startDate / startTime	string? (YYYY-MM-DD / HH:MM)
dueDate / dueTime	string (YYYY-MM-DD) / string? (HH:MM)
dueDays	number? (planned duration)
milestones	Milestone[]?
comments / submissions	TaskComment[] / TaskSubmission[]

Attendance, DailyWorkLog, MediaFile, Notification

Type	Key fields
Attendance	userId, date, checkInTime, checkOutTime?, status (Present/Late/Half Day/Absent), workingHours?
DailyWorkLog	userId, userName, date, tasksDone, problems, notes, attachments[]
MediaFile	name, type (image/video/document), url, size, extension, uploadedBy, dateAdded, projectId?, departmentId?
Notification	userId ('all' or specific), title, message, type, time, read

5. View / Component Catalog

Component	Responsibility
Sidebar	Collapsible nav (icon-only when collapsed), section links, department quick-filter list
Header	Breadcrumb, Ctrl+K search trigger, role switcher, dark/light toggle, notifications dropdown, user card
CommandPalette	Global fuzzy jump across projects/tasks/departments/users, opened via Ctrl/Cmd+K
Dashboards (Admin / Team Leader / Team Member)	Role-specific KPI tiles, charts, shortcuts — swapped based on <code>userRole</code>
DepartmentsView	Three-level drill-down, single view at a time (no split panes): SBU table → project table (per department) → task table (per project) with status dropdown + editable progress
ProjectsView	Portfolio grid of all projects; detail page with Overview / Tasks / Media / Comments tabs
TasksView	Flat task table across all projects; inline status dropdown and progress input; "Create Board Task" drawer with start/due date+time+days fields

CalendarView	Month grid or full 7-day Week view; every day is clickable → modal listing each assigned member's tasks for that date
ReportsView	Aggregate analytics across departments/projects/tasks
UsersView	Personnel directory table (Admin only nav entry); status toggle, password reset action
FilesView	Media library with type filters and upload affordance

Theming

Dark mode is a manual toggle (not OS-driven): `page.tsx` stores the preference in `localStorage['theme']` and toggles a `.dark` class on `<html>`. Tailwind v4's default dark variant follows the OS `prefers-color-scheme`, so `globals.css` overrides it with `@custom-variant dark (&:where(.dark, .dark *))` to make it class-driven instead.

6. Cash Flow Management

Planned — not yet built

Requested addition: a new "Cash Flow" section tracking money coming in (Invoices) against money going out (Bills), mirroring the mockup supplied. This section documents the plan; implementation is on hold pending your sign-off.

6.1 Provided Mockup (recreated)

Cash In			Cash Out		
Income — Invoices			Expenses — Bills		
+ Add Invoice			+ Add Bill		
Account Name	Select an Account (Sales)	Show Existing / Add New	Account Name	Select Cost Account (Admin)	Show Existing / Add New
Dept. (optional)	Select a Dept Type	Show Existing / Add New	Dept. (optional)	Select a Dept Type	Show Existing / Add New
Project (optional)	Select a Project	Show Existing / Add New	Project (optional)	Select a Project	Show Existing / Add New
Status	Received / Not Received	Expected Receiving Day	Status	Paid / Not Paid	Due Date
Attach Photo (optional)			Attach Photo (optional)		

Recreated from the supplied image for inclusion in this document (original file not accessible to embed directly).

6.2 Proposed Data Model

Field	Type	Notes
id	string	
direction	'in' 'out'	Invoice (Cash In) vs Bill (Cash Out)
accountName	string	Linked to a Sales account (in) or Cost/Admin account (out); combobox with "show existing" list + "add new"
departmentId	string?	Optional link to Department

projectId	string?	Optional link to Project
amount	number	Not shown explicitly in mockup — recommended addition so totals can be computed
status	'Received' 'Not Received' (in) / 'Paid' 'Not Paid' (out)	
expectedOrDueDate	string?	"Expected Receiving Day" (in) or "Due Date" (out); shown only while unsettled
attachment	MediaFile?	Optional photo/receipt, reuses existing MediaFile type
createdAt	string	For sorting/history

6.3 Proposed UI Plan

- New top-level Sidebar entry "**Cash Flow**" (Project Management group), routing to a new `CashFlowView.tsx`.
- Two-column layout matching the mockup: **Cash In** (left) / **Cash Out** (right), each with its own "Add Invoice" / "Add Bill" button and its own running table below.
- Add flow opens a form (drawer or inline, consistent with the existing Task creation pattern) collecting: Account Name, optional Department, optional Project, Status, conditional date field, optional photo attachment.
- Account / Department / Project fields behave as "select existing from a list, or add a new one inline" — same combobox pattern already used for Assignee/Project selects elsewhere in the app.
- Table rows below each column list existing entries with status pill and date; clicking Received/Paid vs Not toggles status, consistent with the Task status pattern already implemented.
- Data persists to a new Supabase table (e.g. `cash_flow_entries`) via the Supabase JS client, rather than the legacy localStorage pattern used by the rest of the app today — this is the first data model to land directly on the Supabase backend.

This section is a plan only. Once you confirm the fields/behavior above match what you need, the next step is implementing `CashFlowView.tsx`, the `CashFlowEntry` type in `lib/types.ts`, seed/persistence wiring in `lib/initialData.ts`, and the new Sidebar entry.